

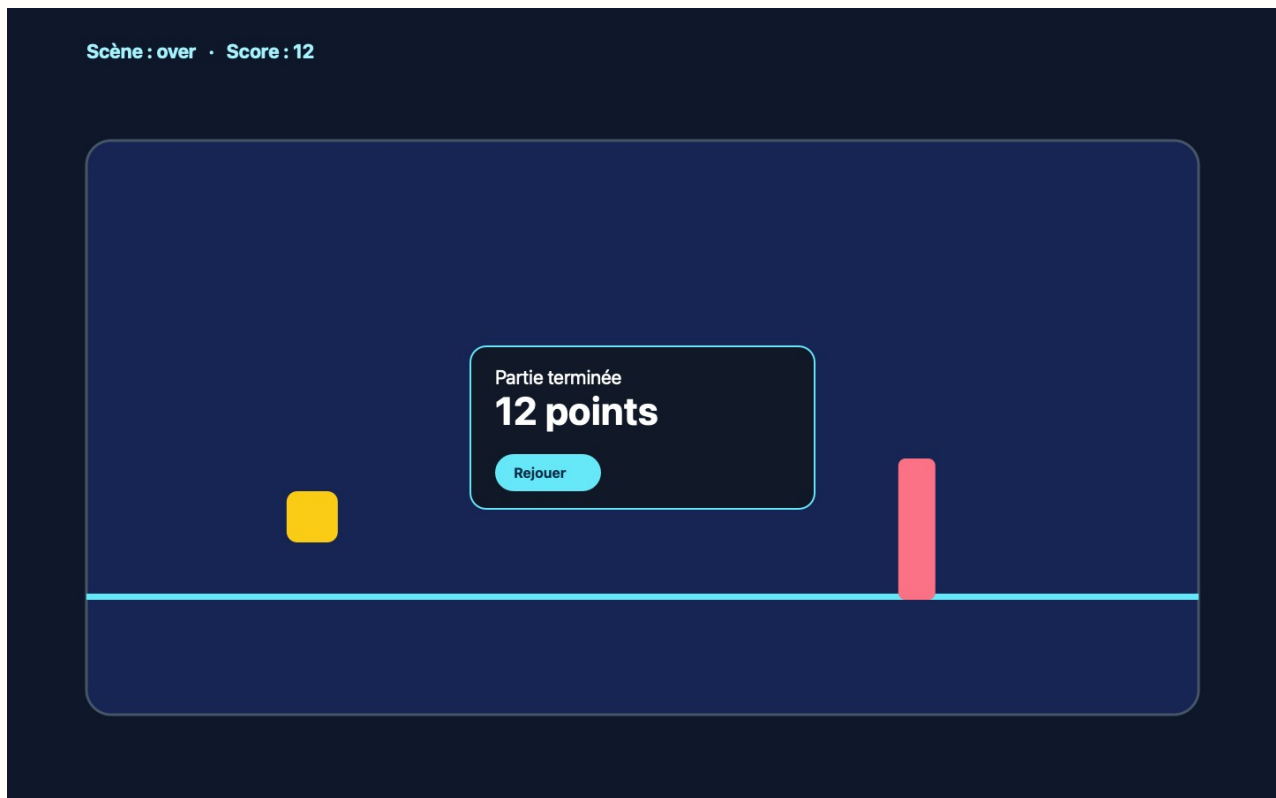
Atelier — Un niveau, trois états

À ouvrir après 1-COURS.pdf du dossier projets/02-niveau-etats/. 2–3 périodes. Le sas POO est requis. Vous repartez du dossier m13/ de l'atelier One-button.

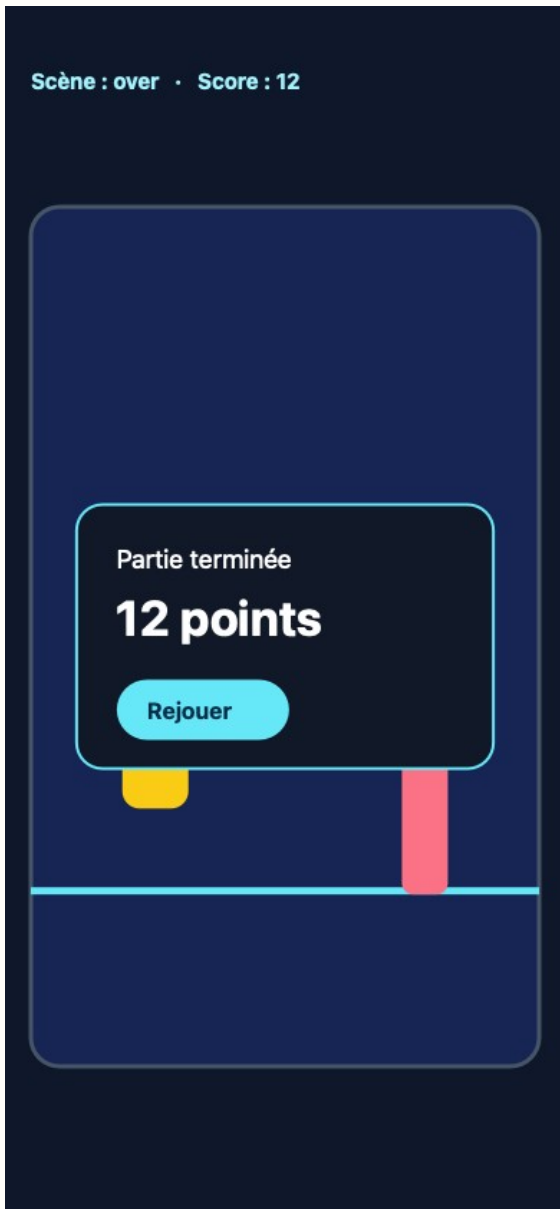
L'idée

Une seule donnée choisit la scène active, et seule la scène de jeu fait avancer le joueur. Le menu attend, le jeu bouge, l'écran final attend une nouvelle décision. Le dessin peut changer : la règle reste **une scène active à la fois**.

À la fin, votre écran ressemble à ceci après une défaite. Le message appartient à l'état over; le niveau reste visible derrière lui.



Sur un écran de 390 pixels, le panneau reste lisible et le bouton Rejouer reste accessible.



Vous gardez exactement cet arbre

```
m13/  
├─ index.html  
├─ concept.md  
├─ css/  
│   └─ style.css  
├─ images/  
├─ js/  
└─ jeu.js
```

Vous n'ajoutez pas trois pages HTML. Les trois états vivent dans le même jeu et utilisent le même bouton.

Étape 1 — Écrire les trois transitions

Dans l'atelier précédent, `menu → jeu` existait déjà. Un niveau complet ajoute une sortie et un nouveau départ. Dans `concept.md`, remplacez la règle précédente par ces trois lignes, adaptées à votre bouton

:

```
menu + action → jeu
jeu + collision → over
over + action → jeu
```

Une **transition** est un changement d'état provoqué par un événement nommé. Ici, `action` et `collision` sont les événements ; `menu`, `jeu`, `over` sont les états.

Étape 2 — Un premier changement avec `changerScene`

Quand plusieurs endroits écrivent directement `etat.scene`, on oublie vite de modifier le texte ou le bouton. La fonction `changerScene` devient l'unique porte de changement : elle reçoit le prochain état, puis met l'écran à jour.

```
function changerScene(prochaineScene) {
  /* à vous : ranger prochaineScene dans etat.scene */
  etiquette.textContent = `État : ${etat.scene}`;
  bouton.textContent = etat.scene === "over" ? "Rejouer" : "Agir";
}
```

Remplacez d'abord seulement le passage `menu → jeu` par `changerScene("jeu")`. Testez ce cas avant de toucher à la collision. Vous devez obtenir le même résultat qu'avant avec une règle maintenant nommée.

Étape 3 — Le joueur contient ses données et son action

Un joueur dont `y`, `vitesseY` et la règle du saut sont éparpillés oblige la boucle à connaître tous ses détails. Une classe rassemble les données et les gestes qui appartiennent au joueur. Complétez seulement la règle **sauter = donner une vitesse vers le haut**.

```
class Joueur {
  constructor() {
    this.y = 220;
    this.vitesseY = 0;
  }

  sauter() {
    /* à vous : donner une vitesseY négative */
  }

  avancer() {
    this.vitesseY += 0.5;
    this.y += this.vitesseY;
  }
}
```

Créez un seul objet avec `const joueur = new Joueur();`. Appelez `joueur.sauter()`, puis `joueur.avancer()` depuis la console et relisez `joueur.y`. Le moule est `Joueur`; l'objet réel est `joueur`.

Étape 4 — Seul `jeu` avance

Une animation qui continue sous le menu ou sous le message final contredit l'état affiché. Dans votre boucle existante, placez l'appel `joueur.avancer()` derrière cette règle nommée :

```
function mettreAJour() {  
  if (etat.scene !== "jeu") return;  
  /* à vous : demander au joueur d'avancer */  
}
```

Testez d'abord en restant au menu : `joueur.y` ne doit pas changer. Passez ensuite à `jeu` : la valeur doit évoluer. Vous vérifiez ainsi l'idée sans attendre que tout le niveau soit fini.

Étape 5 — Une collision termine le niveau

Choisissez une limite simple, par exemple le bas de la scène. La règle **estPerdu = le joueur dépasse la limite** appartient au joueur parce qu'il connaît sa position.

```
estPerdu(limite) {  
  /* à vous : retourner vrai quand this.y dépasse limite */  
}
```

Après `joueur.avancer()`, demandez `joueur.estPerdu(hauteurDuSol)`. Si la réponse vaut vrai, appelez `changerScene("over")`. Ne testez pas le texte du bouton et ne cachez pas la transition dans le dessin.

Étape 6 — Rejouer remet les données à zéro

Rejouer ne signifie pas seulement masquer le panneau : la position et la vitesse doivent aussi repartir ensemble. Ajoutez une méthode `reinitialiser()` au joueur, puis appelez-la avant `changerScene("jeu")` lorsque l'état courant vaut `over`.

Testez le trajet complet : menu → jeu → collision → over → jeu. À chaque arrêt, dites l'état courant avant de cliquer.

Réussi si

- [] `concept.md` nomme les trois transitions.
- [] `changerScene()` est le seul endroit qui écrit `etat.scene` après l'initialisation.
- [] Joueur contient la position, la vitesse et les actions du joueur.
- [] La boucle n'appelle `avancer()` que dans l'état `jeu`.
- [] `estPerdu(limite)` porte une règle lisible et testable.
- [] Rejouer remet les données à zéro avant de relancer.
- [] La page se lit à 1440 et à 390 pixels.

AVEC L'IA

Montrez seulement `mettreAJour()` et `changerScene()` à une IA. Demandez-lui de décrire le trajet menu → jeu → over en cinq phrases, sans proposer de nouveau code. Vérifiez chaque phrase dans vos transitions.

POUR LES RAPIDES

Ajoutez l'état pause. Il gèle le joueur comme menu, mais conserve sa position. Écrivez la transition dans `concept.md` avant d'ajouter la branche.

Livrable

Rendez le même dossier `m13/`, avec `concept.md` mis à jour. À l'oral, montrez un objet créé avec `new Joueur()`, puis expliquez pourquoi seul l'état jeu appelle `avancer()`.